

M03 / STUDY COMPANION

M03 Quick Revision Book

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 5

How to revise 5

DE03-01 | INNER JOIN and LEFT JOIN 5

60-second recall 5

Recall task 5

DE03-02 | Cardinality, row multiplication and join validation 5

60-second recall 5

Recall task 5

DE03-03 | UNION and UNION ALL 6

60-second recall 6

Recall task 6

DE03-04 | String functions for cleaning and standardization 6

60-second recall 6

Recall task 6

DE03-05 | Dates, timestamps and time analysis 6

60-second recall 6

Recall task 6

DE03-06 | Subqueries and nested logic 7

60-second recall 7

Recall task 7

DE03-07 | CTEs and readable multi-step SQL 7

60-second recall 7

Recall task 7

DE03-08 | Window functions and PARTITION BY 7

60-second recall 7

Recall task 7

DE03-09 | ROW_NUMBER, ranking and LAG/LEAD 8

60-second recall 8

Recall task 8

DE03-10 | Advanced aggregation and multi-grain reasoning 8

60-second recall 8

Recall task 8

DE03-11 | PostgreSQL setup, schemas and dialect awareness 8

60-second recall 8

Recall task 8

DE03-12 | Constraints and transactions 9

60-second recall 9

Recall task 9

DE03-13 | Indexes and EXPLAIN basics 9

60-second recall 9

Recall task 9

DE03-14 | Views and reusable database logic 9

60-second recall 9

Recall task 9

DE03-15 Integrated unseen SQL challenge	10
60-second recall	10
Recall task	10
DE03-16 Timed SQL interview set and explanation	10
60-second recall	10
Recall task	10

Read this first

QUICK REVISION BOOK

Use this only after you have studied the learner book. Each lesson keeps the canonical language and core proof, plus one recall task. This is not a replacement for hands-on practice.

Coverage: 16 canonical lessons

Route: learner book -> practice -> revision -> closed-book test -> protected answers -> changed retry.

How to revise

Read the lesson card, close the page, explain it in your own words, then answer the recall task. If recall is weak, return to that lesson only - not the entire module.

DE03-01 | INNER JOIN and LEFT JOIN

60-second recall

INNER JOIN keeps matching pairs. LEFT JOIN also preserves unmatched left rows. Use the relationship key, qualify ambiguous names and predict multiplicity. ON defines matches; WHERE can remove rows after the join. Record both row counts and unmatched cases, not only a familiar-looking name.

[Refresh source: Learn PostgreSQL Tutorial - Full Course for Beginners](#)

Recall task

Return every order but attach a customer name only when the customer belongs to North. Keep all original orders, including those outside North and O106. Explain why WHERE region = 'North' would fail that preservation rule.

DE03-02 | Cardinality, row multiplication and join validation

60-second recall

Declare both input grains, matching keys, expected cardinality and final grain. Count source rows, joined rows, represented keys and unmatched cases. Aggregate line measures to order grain before adding order-level charges. DISTINCT on a money value cannot replace a key-aware repair.

[Refresh source: Learn SQL Beginner to Advanced in Under 4 Hours](#)

Recall task

Count all rows from a LEFT JOIN of orders to items. Also count non-null item line numbers and distinct order IDs. Explain why these three counts differ, and identify the row causing the difference between the first two counts.

DE03-03 | UNION and UNION ALL

60-second recall

JOIN combines matching records side by side; UNION operations append compatible result rows. UNION ALL preserves occurrences, UNION removes duplicates of the whole selected row. Align column meaning and type. Preserve source labels when provenance matters, and distinguish legitimate repeated values from replayed business events.

[Refresh source: Learn SQL Beginner to Advanced in Under 4 Hours](#)

Recall task

Find categories appearing in `course.retry_sales` on both November 1 and November 2 using INTERSECT. Then find categories from November 1 absent on November 2 using EXCEPT.

DE03-04 | String functions for cleaning and standardization

60-second recall

Keep raw data and describe each cleaning rule. `NULLIF(TRIM(...), '')` can represent blank labels as missing. Count raw rows, usable values and unique cleaned values separately. Pattern matches identify candidates, not reliable product identity. Do not erase unknown mappings or leading zeros to make data look tidy.

[Refresh source: Learn SQL Beginner to Advanced in Under 4 Hours](#)

Recall task

Using only the three literal inputs below, calculate raw value, cleaned value and cleaned length: 'Bag ', 'BAG' and ' '. You can use UNION ALL to create three SELECT statements. Decide whether a blank cleaned value should have length zero or a null length under the chosen missing-label rule.

DE03-05 | Dates, timestamps and time analysis

60-second recall

A calendar date, local clock reading and global instant are different things. State the reporting zone, convert before extracting a local date, and use inclusive-start/exclusive-end windows. Test a record at the beginning, just before the end and exactly at the next beginning. Reconcile grouped counts to the included events.

[Refresh source: SQL for Data Analytics - Learn SQL in 4 Hours](#)

Recall task

Select September 1 according to UTC instead of India. Then explain why a different pair of IDs is correct despite the stored events being unchanged.

DE03-06 | Subqueries and nested logic

60-second recall

A correlated subquery uses the current outer record. EXISTS preserves outer grain while testing any match; NOT EXISTS tests no match. Check that correlation is present and that absence means exactly what the business sentence says. A scalar position requires one value, not an arbitrary list of records.

[Refresh source: Learn SQL Beginner to Advanced in Under 4 Hours](#)

Recall task

Find customers with at least two orders using a scalar correlated COUNT. Return ID and name, one row per customer. Explain why the count cannot be replaced by EXISTS alone.

DE03-07 | CTEs and readable multi-step SQL

60-second recall

WITH names intermediate results within one statement. State each step's grain, aggregate before joining across grains, and preserve required headers with LEFT JOIN. Justify zero fallbacks using the dataset's completeness rule. Validate unique output keys, empty cases and independently scoped totals.

[Refresh source: Learn SQL Beginner to Advanced in Under 4 Hours](#)

Recall task

Use item_totals to identify order headers with no matching item summary. Return their order IDs and explain why you test the right-side key rather than item_gross = 0.

DE03-08 | Window functions and PARTITION BY

60-second recall

Window aggregates can preserve detail grain. PARTITION BY defines calculation groups; an ordered frame defines contributing rows. WHERE affects rows before the window. Use an outer query to filter after computing a full-population window. Repeated partition totals are not additive detail measures.

[Refresh source: Learn SQL Beginner to Advanced in Under 4 Hours](#)

Recall task

Produce a running gross total for course.retry_sales by ordered_on, breaking ties with order_id. Use an explicit ROWS frame and predict the total after R202.

DE03-09 | ROW_NUMBER, ranking and LAG/LEAD

60-second recall

Use ROW_NUMBER to choose one record with a deterministic tie-breaker. Use RANK or DENSE_RANK when equal measures should share positions. Filter window results in an outer query. LAG/LEAD follow row order, not an assumed calendar. Preserve the meaning of missing predecessors and unidentified partition keys.

[Refresh source: Learn SQL Beginner to Advanced in Under 4 Hours](#)

Recall task

For course.retry_sales in date/ID order, return each amount, its previous amount and the difference from the previous amount. Preserve the first difference as NULL. Explain why filling that first difference with zero would be an added assumption.

DE03-10 | Advanced aggregation and multi-grain reasoning

60-second recall

Track grain and key at every stage. Aggregate line measures before combining header measures. Preserve explicitly required empty and unknown cases. Reconcile counts and measures at equal scopes. Combine group totals and counts for a weighted mean; do not blindly average averages.

[Refresh source: Learn PostgreSQL Tutorial - Full Course for Beginners](#)

Recall task

Using the same item-to-order preparation, report the count and average gross for non-empty orders only. Independently calculate total gross divided by that order count. Explain the difference from the all-order average in the guided report.

DE03-11 | PostgreSQL setup, schemas and dialect awareness

60-second recall

Know your connection's server, database and role. A schema is a namespace, and qualification avoids search-path ambiguity. Metadata reveals types and nullability. CREATE defines an object, INSERT adds records, SELECT reads them. Translate dialects by meaning and verify the result; do not install a new database for every video.

[Refresh source: SQL for Data Analytics - Learn SQL in 4 Hours](#)

Recall task

Create a temporary table called desk_retry with text reference as primary key and a non-null integer units column. Insert X01 with units 3, then read it. Explain whether merely saving those statements to a file creates the table.

DE03-12 | Constraints and transactions

60-second recall

Use separate constraints for identity, missingness, range and relationships. CHECK does not automatically forbid NULL. BEGIN opens a transaction, COMMIT accepts it and ROLLBACK cancels its uncommitted work. Recover an aborted transaction with ROLLBACK and then fix the first error. Constraints do not replace source-quality checks or a clear business contract.

[Refresh source: Learn PostgreSQL Tutorial - Full Course for Beginners](#)

Recall task

In a new temporary stock table, insert ONE/5. Begin a transaction, reduce units to 4 and COMMIT. Verify 4 remains. Attempt a second insertion with the same key and record the error without removing the original record. Explain why ROLLBACK after the successful commit cannot restore the earlier 5.

DE03-13 | Indexes and EXPLAIN basics

60-second recall

Indexes offer access paths with storage and write costs. EXPLAIN shows a plan; EXPLAIN ANALYZE executes and measures. Compare estimated and actual rows, but do not read planner cost as milliseconds. Check a narrow and a broad query, preserve correctness, and report what your environment actually did.

[Refresh source: Lesson 3, PostgreSQL Indexes and B-Trees \(and an introduction to EXPLAIN\)](#)

Recall task

Inspect a query selecting customer_no = 21, for which no customer_no index has been created. Predict the number of matching rows, execute a COUNT to check it, and inspect the plan. Explain why an index on order_no does not automatically become an index on customer_no.

DE03-14 | Views and reusable database logic

60-second recall

An ordinary view stores a query definition, a materialized view stores results, and a CTE is scoped to one statement. Temporary views are session-scoped. Publish clear grain, meaning, field and null-handling contracts; test counts and totals. A reusable name does not guarantee correctness, security, ordering or speed.

[Refresh source: PostgreSQL Tutorial Full Course 2022](#)

Recall task

Using the disposable view_source table and view_demo, begin a transaction, change units from 3 to 4, inspect the view, then roll back and inspect it again. Explain why the view follows the transaction's visible source state.

DE03-15 | Integrated unseen SQL challenge

60-second recall

For an unfamiliar source, inspect its fields and grain before writing the final report. Construct a validated order-level result, then summarise or rank it. Empty orders may still have fees. Repeated equal fees are legitimate on different orders. Save evidence of independence and repair, not just a polished final query.

Recall task

Use `course.challenge_retry_customers`, `course.challenge_retry_orders` and `course.challenge_retry_items`. Build all-order totals and region totals from a blank editor. Keep empty orders and unknown identity, reconcile counts and amounts, and calculate the average gross per order over the complete order population. Explain why averaging region averages equally would be unsafe. Do not reuse the previous result numbers.

DE03-16 | Timed SQL interview set and explanation

60-second recall

Interview practice measures correct reasoning under an agreed constraint, not typing speed alone. State grain, null policy, tie rules and scope, then verify actual outputs. Keep a timed attempt honest about assistance. After this lesson and its repairs, attempt G02; M03 completion alone does not automatically pass the gate.

Recall task

Use `course.practice_sales` for two fresh interview questions: choose the highest-gross sale per category, resolving ties by ascending ID; then return categories whose known-paid gross exceeds 50. Explain when `WHERE` happens relative to `GROUP BY` and a window calculation. For a third task, use `course.challenge_retry` tables to prove the all-order total and state its average denominator. Save the time and assistance record separately from the first set.